



Technical University of Denmark

Deep learning for automatic sleep classification in
electroencephalography

Pedro Vieira, s240181

May 5, 2025

Contents

1	Abstract	1
2	Introduction, motivation and literature review	1
2.1	Motivation and objectives	1
2.2	Alzheimer’s disease	1
2.3	Neurostimulation	1
2.4	Sleep stages	1
2.5	Automatic sleep staging	1
2.6	DRAFT	1
3	Methods	1
3.1	Original pipeline	1
3.1.1	Network architecture	2
3.1.2	Steps of the pipeline	3
3.2	Modifications to the pipeline	3
3.3	Additions to the pipeline	4
3.4	Pipeline features	5
4	Results and discussion	6
4.1	Sleep staging results	6
4.2	Multi-rater comparison	7
4.3	Validation with ground truth dataset	9
5	Conclusion	9
5.1	????	9
5.2	Shortcomings of our approach	9
5.3	Future prospects	9
5.4	Objectives acheived ??	9
	Acknowledgments	9
	References	10

1 Abstract

2 Introduction, motivation and literature review

2.1 Motivation and objectives

The final objective of any medical intervention is to treat or prevent a disease, or at least to better the patient's quality of life. To achieve this, it is necessary to study the underlying disease etiologies and pathophysiological mechanisms, in order to modify or interrupt the course of the disease to produce the healing outcomes desired or block unwanted ones.

With this in mind, and in the light of new research that links Alzheimer's disease regression with neurostimulation as well as good quality deep sleep [**<empty citation>**], one goal of this project was to better understand these mechanisms, and how they relate to one another, and to the pathophysiology of Alzheimer's disease.

The main technical goal of this project is to build upon and apply a preexisting automatic sleep staging EEG pipeline, to eventually be used internally for a streamlined sleep staging setup. The goal is to eventually allow Optoceutics to quantify the amount of deep sleep that users of their product are getting, possibly to modify their product or treatment.

2.2 Alzheimer's disease

2.3 Neurostimulation

2.4 Sleep stages

2.5 Automatic sleep staging

2.6 DRAFT

$$\mathcal{S} = \{W, N1, N2, N3, REM\} \quad (1)$$

Predictions For each time step

$$\mathcal{Y} = \left\{ y \in \mathbb{R}_+^K \mid \sum_i y_i = 1 \right\} \quad (2)$$

$$\mathcal{Y} = \left\{ y \in \mathbb{R}_+^K \mid \sum_i y_i = 1 \right\}, K = |\mathcal{S}| \quad (3)$$

$$\operatorname{argmax} y : \mathcal{Y} \rightarrow \mathcal{S} \quad (4)$$

3 Methods

3.1 Original pipeline

As mentioned, most of the work on this project was built on top of a automatic deep learning sleep staging algorithm and pipeline developed in [2]. The algorithm was tested and validated on thousands of annotated sleep staging trials, from multiple cohorts, and trained in a number of different settings (like training with some cohort's data and validating on the others). Because the paper also provided the learned weights for the best model, these weights will be used for the predictions in our own data without much validation of their accuracy, especially because we lack annotated sleep staging data of our own to validate the model against.

The machine learning task at hand was to predict the sleep stage (wake, N1, N2, N3 or deep sleep, and REM sleep) of each 30 second window of time-dependent polysomnography (PSG) data. Because the data and the predictions are both time dependent, the goal was to infer, through machine learning, a function that could map PSG data to the corresponding sleep stage for that time window (or epoch).

3.1.1 Network architecture

The architecture of the network used in [2] was inspired by ResNet-50 and adapted to 1-dimensional, time-dependent signals for classification tasks. A deep learning approach was used because it doesn't require preprocessing or manual feature extraction, and the mapping function is automatically learned end to end. An overview of the network architecture can be found in Fig.1. The training setup used the default parameter values of $C = 4$, $f_s = 128$ Hz, $T = \tau f_s$, $K = 5$, $R = 7$, and $f_0 = 4$ for the number of input channels, sampling frequency, the sequence length, the number of sleep stages, the number of consecutive residual blocks, and the base filter kernel size, respectively, with τ being the number of successive 1 second predictions per epoch, usually just 30, but tested with other divisors of 30.

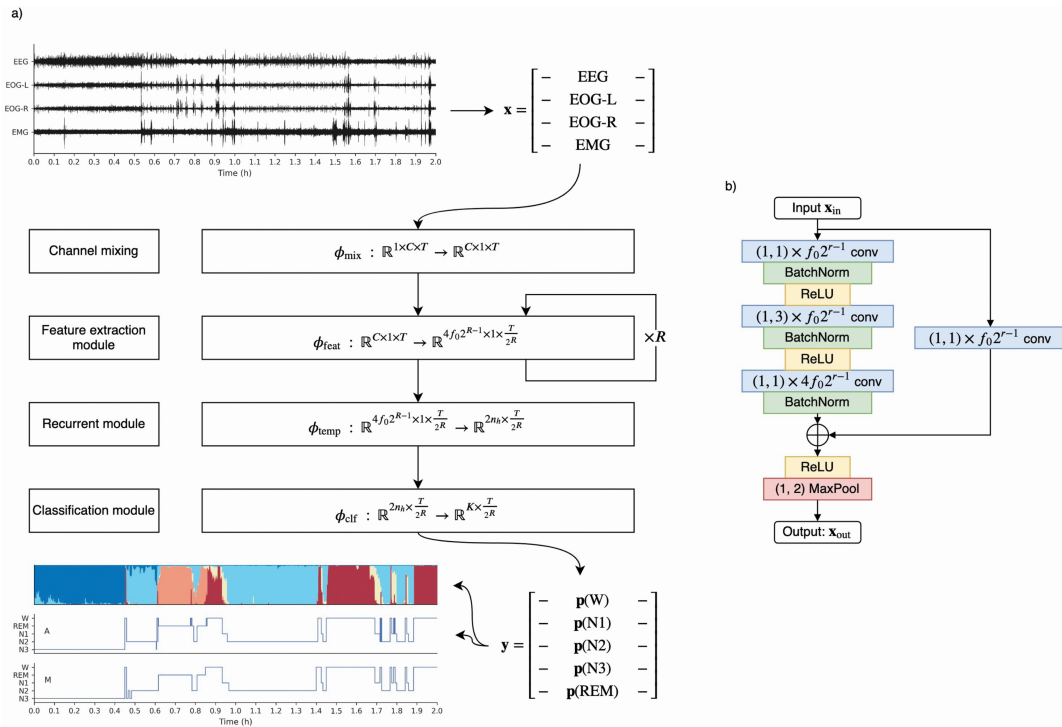


Figure 1: Network architecture used in [2]

Essentially it comprises:

- **A channel mixing module**, that changes the dimensions of the data to fit the latter part of the network, and introduces non-linear mixing between the channels' data.
- **A feature extraction module**, that comprises R residual blocks, each using 2D convolutions to reduce the increase again the number of feature maps, batch normalizations and activations. Each block is simple, ensuring good conversion, and adding a lot of blocks give more expression to the network.
- **A temporal processing module**, that uses Gater Recurrent Units to introduce "past" and "future" dependencies on the data, mimicking how a human annotator would analyse the data.

-
- **A classification module**, that essentially performs a softmax activation on the results of the previous layer, giving the predicted probability of each sleep stage for each instant. The argmax of the results of the softmax for each instant give us the predicted stage of sleep for that instant.

3.1.2 Steps of the pipeline

A lot of the code used for this project, as well as the learned weights for the best model of the paper were made public through a GitHub repository [2]. Instructions on how and what to run in the pipeline are also available. Some essential steps in the pipeline are:

- **Setting up the .edf files** to be read in a folder and modify a config file to let the pipeline know the new location of the data.
- **Latel identification**, by running a simple script that lets us map each electrode in our data to each electrode required by the pipeline, because there might not be a 1:1 correspondence between these.
- **Generate .h5 files from the data**, by running another script. Including modifying the code to extract the relevant patient metadata and data from our edf files. This step generates one h5 file for each subject, that includes the polysomnography data and ground truth hypnogram (if one was provided), as well as a .csv file for the cohort, with subject ID and file path information for each subject.
- **Inference on new data**, by calling the predict function with the learned weights on the h5 files, mentioning another config file. This generates .pkl files with the predicted sleep stage for each timestamp in the the PSG data, and performs some comparative metrics with the labeled hypnogram (if one was provided).

3.2 Modifications to the pipeline

The first step of coding for this project was to fork the original GitHub repository [2] to keep all of the original content while allowing me to develop on top of that, keeping the coherence and structure. However some modifications had to be made on the code, and some involved copying script files and hard coding some modifications on them, in order to fit some incompatibilities between the pipeline and our data and objectives for the project.

First, it is important to mention that the model was trained on polysomnography data. Meaning that it includes scalp EEG electrodes, as well as chin EMG and eletrooculogram (EOG). On the other hand, the data we have been acquiring is only from EEG channels. Because of that, when mapping the channels from our data to the channels in the pipeline, the chin EMG and and EOG were left black, meaning that, after preprocessing of the channels, 1 out of the 4 data values that was passed to the model for each timestamp was zeroed out.

Another important incompatibility is that all throught the pipeline, it expects there to be a valid labeled hypnogram for the corresponding .edf file. However, the goal of this project was to perform predictions on new, unlabeled data, and comparing to the predictions from another pipeline to assess their agreement, not comparing with the ground truth (which was not available, as it would require a trained technician and a lot of time to annotate). To work around this incompatibility with the pipeline, 2 of the python files were modified to **1:** produce a blank (all zeros) dummy hypnogram with the same size as the EEG data (instead of reading th hypnogram data from another .edf file) and **2:** ignore the comparison between predicted and "true" hypnogram at the end of the code, providing only the predictions, that would be compared to the results from other pipelines later.

3.3 Additions to the pipeline

On top of the prediction pipeline, a few postprocessing and visualization scripts were made, to better visualize the results, manipulate the data and compare to other pipelines. These files, all located in a subfolder on the repository, are to be run as python scripts or modules, and include the `-h` flag and some other configurations to be called when running the code.

The scripts created are:

- `display_hypnogram <predictionspath> --eval-window 30 --begining-index 0`: that simply manually retrieves one `predictions.pkl` file (given in `predictionspath`) and plots the predicted hypnogram, as well as the distribution of the predicted relative probabilities of each sleep stage for each timestamp. The hypnogram is plotted using Yasa's `plot_hypnogram` function [3]. The eval-window can be change to perform averaging over the second-wise predictions in a smaller or larger window, the default, and most commonly used in the literature, is 30 (windows of 30 seconds). The time series of predictions is divided into chunks with size equal to the window size, and the predicted sleep stage for each chunk is the sleep stage with the highest probability inside all timestamps of that chunk of time. A larger window makes the output smoother but also loses time resolution. The beginning index can also be changed, if, for example, the beginning of the data corresponds to an awoken state, is too noisy or unpredictable and could be discarded for visualization.
- `predictions_to_txt --eval-window 30 --config <config>.yaml --prediction-dir <predictdir> --output-dir <outputdir>`, which, for every file in the specified directory, converts the predictions into a `.txt` file in a lightweight and easy-to-read format, to be compared to other pipelines. This file format has one row for each epoch, with each row containing the predicted sleep stage. This step was included to make comparing results with other pipelines easier and streamlined, as the `sleepEEGpy` sleep staging wrapper [1] exports the predicted hypnogram in this format. This function has a number of keyword arguments:
 - `--eval-window`, that does the same as explained for the previous function (controlling the size of each epoch), defaulted to 30.
 - `--config`, references the `.yaml` file that was also used for obtaining the predictions in the original pipeline
 - `--prediction-dir`, with the directory of all the `.pkl` predictions files, to be converted into the simple `.txt` format.
 - `--output-dir`, with the directory of where the `.txt` files should be placed.
- `hypnogram_comparison --folder <folderpath>`, which performs multi-rater analysis on multiple sleep staging predictions (from multiple different pipelines), each produced independently from the same `.edf` sleep EEG data. This function receives the folder of the `.txt` to be analyzed files as input. The `.txt` files presented can have different sizes (if for example 2 different window sizes were used for the same data), and the code will repeat the predictions of each file the number of times necessary for their duration to match (essentially make each file the size of the least common multiplier between their original sizes). This script produces some plots:
 - A confusion matrix for pairwise comparison of the predictions made by 2 of the raters for each timestep.
 - A hypnogram comparison between which sleep stage each rater (each pipeline) has predicted for each time step, visualized as all hypnograms' time series superimposed as a function of time.

- A barplot with the proportion of number of agreeers for each time stamp. Essentially telling, on how many timestamps do all the models agree.
- A matrix of inter-rater agreement (accuracy), essentially saying how much each pair of raters agree with each other.
- A matrix of inter-rater Cohen's kappa score, essentially the same as the previous, but with the Cohen's kappa instead of agreement.

3.4 Pipeline features

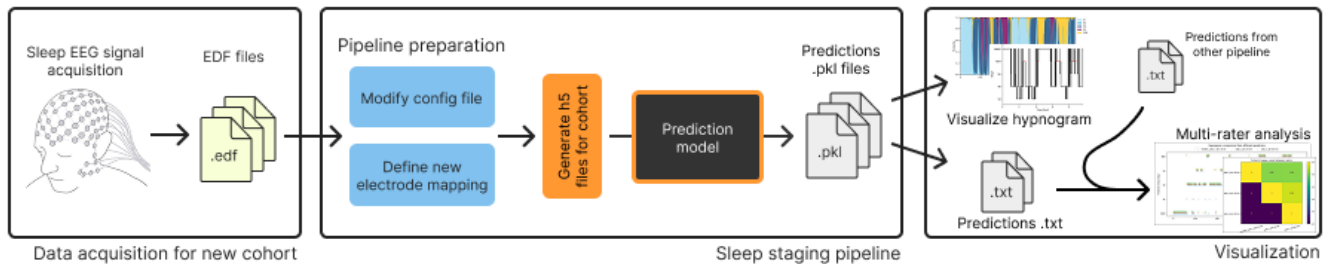


Figure 2: Simplified diagram of the full pipeline

With this, we arrive at the final pipeline developed for this project, essentially consisting in a wrapper around the sleep staging pipeline in [2]. Fig.2 includes a schematic of the full pipeline. The 3 big blocks are essential independent and almost isolated, and should be applied in sequence. Ensuring readability and ease of use for the posterity was also taken in consideration, in case automatic sleep staging continues to be of interest to Optoceutics and as more data is acquired through the years.

The sleep data can be recorded through time with the protocol being used by Optoceutics and the research, and introduced into the pipeline as more and more data is gathered. The pipeline preparation steps only need to be done once for a new cohort of data, with cohort meaning a set of EEG .edf files with the same electrode setup and recording settings. As more data is introduced, generating the cohort files and passing them through the sleep staging prediction model is done by running 2 commands in the local terminal. The visualization part of the pipeline allows for manual visualization of the prediction results as well as small processing and comparison with the predictions from other pipelines, for a multi-rater assessment.

The parameters used on the pipeline preparation block for this project assumed the data consisted of a simple EEG recording, not a full polysomnography, to ensure consistency with the recordings we were provided. However, this results in loss of information, and most certainly in worse prediction results as well. If Optoceutics eventually acquires full polysomnography data, running the electrode mapping function again would ensure the information is not lost. Additionally, the prediction model could be swapped by another as long as they take the same input and output. Finally, the multi-rater comparison can essentially be done with any number of other pipelines, as long as they can be converted into the aforementioned .txt format. The analysis can also be done with manually labeled hypnograms if they are eventually obtained for Optoceutics' sleep recordings, meaning the prediction model or the data gathering could be validated.

Finally, this project was done with a lot of collaboration from a colleague, who assessed the quality of another automatic sleep staging pipeline, using sleepEEGpy [1], a wrap around of another sleep staging python library: Yasa [3]. The multi-rater analysis part of this pipeline was mostly used to compare the results from each of the 2 pipelines, and because labeled sleep EEG data was unavailable, agreement

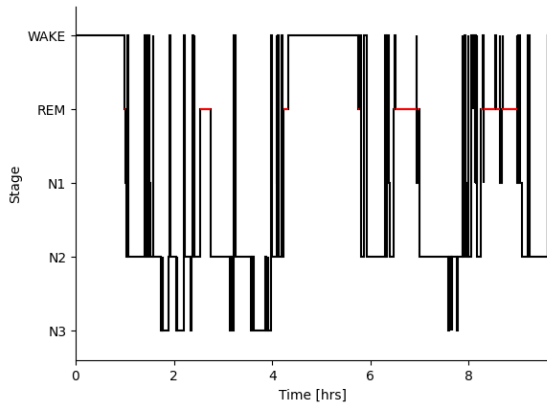
between the raters was considered as a good enough proxy for the ground truth. In fact, as one the goal of the project was to expand Optoceutics' library of labeled sleep EEG data, and to assess the feasibility of further sleep studies, this multi-rater setup was more than enough.

4 Results and discussion

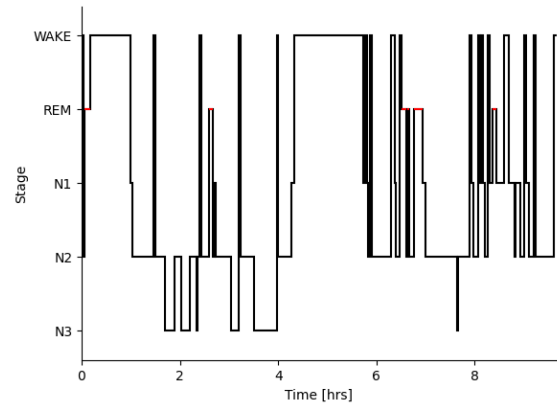
In total, only 2 overnight EEG sleep recordings (from 2 different subjects DESCRIBE THEM???) were used in the pipeline during the course of this project. However, as was described, the pipeline is prepared to receive more data, and obtaining the predictions for them is streamlined. Reporting on all sleep recordings is, therefore, slightly redundant, as the goal was not to obtain good results for one patient, but streamlining obtaining those results for a cohort of subjects fast and efficiently. With this, only the results for one of the subjects will be shown on most plots, to be used as an illustrative example of the pipeline's functioning and output formats.

4.1 Sleep staging results

From the `visualize_hypnogram` script, the hypnogram (Fig.3) and prediction probabilities plot (Fig.4) are produced based on the `predictions.pkl` files for each subject, which are the output of the pipeline in [2].



(a) Hypnogram with window of 30 epochs



(b) Hypnogram with window of 120 epochs

Figure 3: Predicted hypnograms for the same data, for 2 window sizes

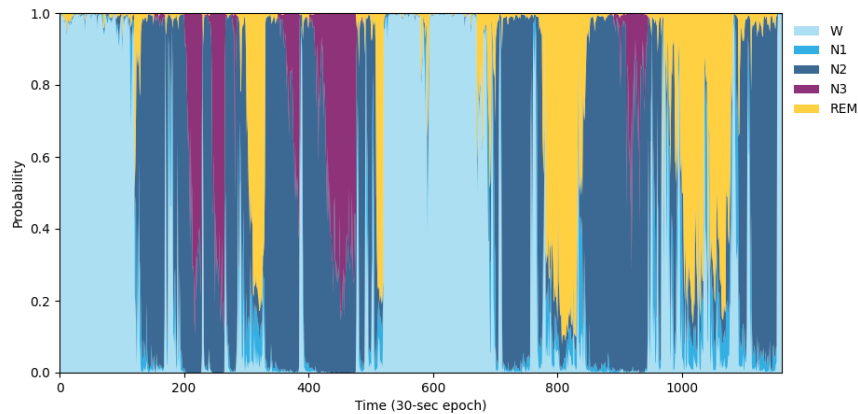


Figure 4: Probabilities for each sleep stage for each epoch

The results from Fig.3a and Fig.3b were obtained by running the `display_hypnogram` twice, with different window sizes. A initial index of 18000 was also used, as the recording had a few hours where the subject was still awake. As expected, the result with a bigger window (2 minutes) is smoother but loses a lot of time resolution. Namely, the amount of predicted REM sleep is almost halved with a bigger window. This stage of the sleep cycle can sometimes be very short (5 minutes), and the bigger window might have a harder time picking it up.

On the other hand, Fig.4 shows the predictions' confidence of each sleep stage for each 30 seconds epoch. In essence, each point in the hypnogram in Fig.3a is the sleep stage with the highest probability for that 30 seconds epoch from the predicted hypnogram probabilities (Fig.4).

4.2 Multi-rater comparison

For the multi-rater comparison of results from different pipelines, it was imperative that the predictions from each were in the same format. Because the results from the pipeline used by my colleague ([3, 1]) automatically exported the predictions in the txt format mentioned (with the prediction for each epoch in each line), running the `predictions_to_txt` on the predictions from this pipeline would ensure coherence. Like for visualizing the hypnogram, the `predictions_to_txt` script was run twice with 2 different values for the evaluation window: 30 and 60 seconds. After placing these 2 txt files (`subj-1_win-30.txt` and `subj-1_win-60.txt`) as well as the one produced from the other pipeline (`model2_subj-1_win-30.txt`), all 3 obtained from the same sleep EEG data, in the same folder, running the `hypnogram_comparison` script produced the plots below. It is important to mention that, as the first two files were produced from the same predictions but with different sizes of prediction windows, the results between these two were expected to be quite similar, and were included side to side as a form of control.

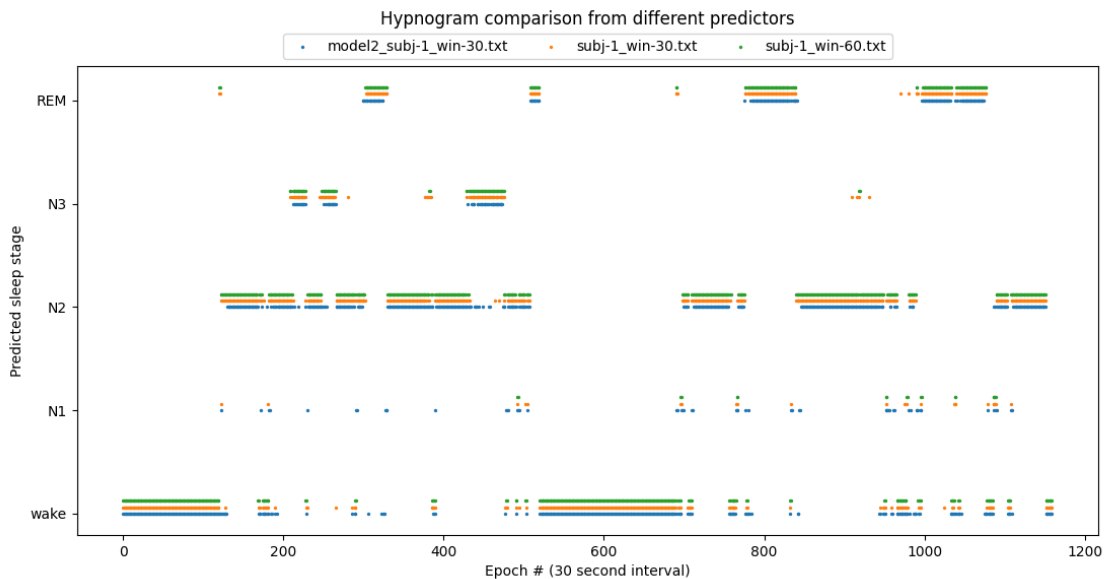


Figure 5: Multi-rater hypnogram predictions

Confusion matrix of predictions between models 1 and 2, for subject 1

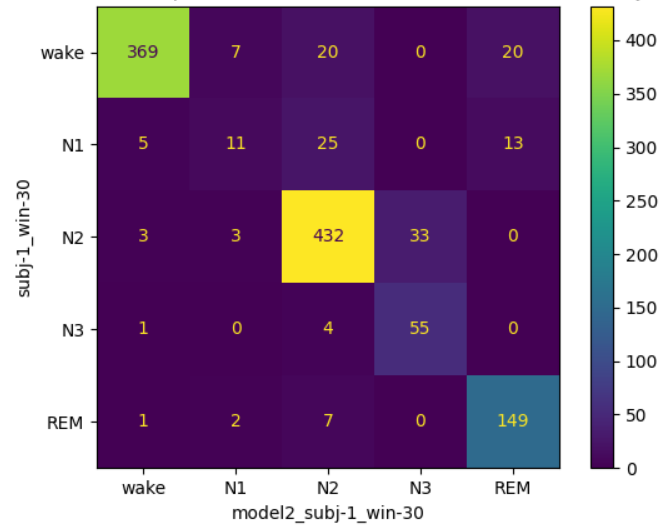


Figure 6: Confusion matrix for all epoch predictions between model2_subj-1_win-30 and subj-1_win-30

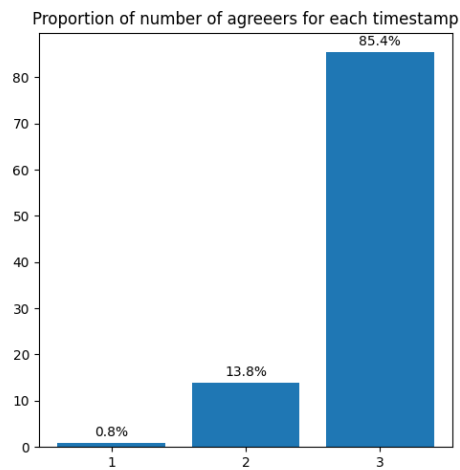


Figure 7: Percentage of time stamps where none, 2 or all 3 prediction models agree

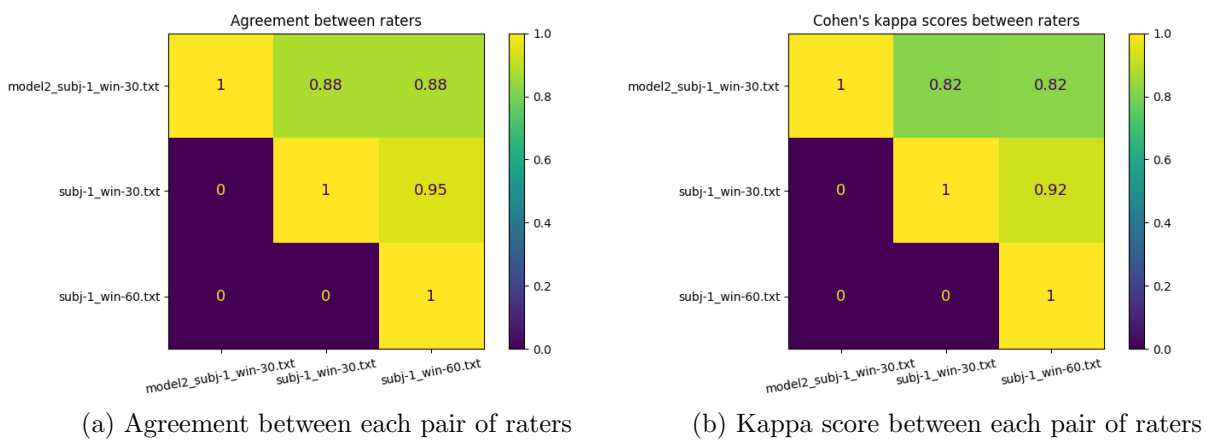


Figure 8: Agreement and Kappa scores between each pair of raters

In Fig.5, the superimposed predicted hypnogram for each of the 3 models is shown. Together with the confusion matrix of predictions between `model2_subj-1_win-30` and `subj-1_win-30`, we show that these two models agree on most pairs of predictions, most notably in the wake and REM stages. Crucially, there seems to be less agreement in the light and deep sleep, with model 1 [2] predicting more light sleep (N1) and model 2 [3] predicting more deep sleep, in general.

Due to the lack of real annotated data, the number of raters that agree on the predicted sleep stage for a certain time stamp is a good proxy for the confidence on the prediction of the ensemble. And, if all raters, i.e. all automatic sleep staging algorithms used, agree on a certain prediction, we could, to a certain degree of confidence, assume that as the true predicted label for that time stamp. Fig.7 shows that on 85% of the time, all 3 predictions agree, which is already a good sign that the predictions are not far from the truth.

Fig.8a shows the percentage of agreement between each pair of models. Likewise, Fig.8b, shows the Kappa score between each pair of model predictions. The diagonal of the matrix is, by definition, just ones, and the lower triangular matrix was displayed as zeros to avoid confusion, but the matrices are essentially symmetrical because both agreement and Kappa score are invariant to changing the order of the pair of models.

As expected, the agreement and Kappa score are almost 1 between `subj-1_win-30` and `subj-1_win-60` (0.95 and 0.92 for agreement and kappa, respectively). Both metrics were also the same between any of the predictions with different window sizes from model 1 and the predictions from model 2 (0.88 agreement and 0.82 kappa score). With all this, the multi-rater ensemble-like setup demonstrated quite good levels of agreement between multiple models, and proved to be a satisfactory automatic sleep staging algorithm.

4.3 Validation with ground truth dataset

5 Conclusion

With all this, and given the lack of a true label on the sleep EEG data acquired, the multi-rater ensemble-like setup is not only satisfactory as a quick and effective automatic classification setup, but also demonstrated quite good levels of agreement between multiple models.

5.1 ?????

5.2 Shortcomings of our approach

5.3 Future prospects

5.4 Objectives achieved ??

Acknowledgments

References

- [1] G Belonosov et al. “SleepEEGpy: a Python-based software wrapper package to organize preprocessing, analysis, and visualization of sleep EEG data”. In: *bioRxiv* (2023), pp. 2023–12.
- [2] Alexander Neergaard Olesen et al. “Automatic sleep stage classification with deep residual networks in a mixed-cohort setting”. In: *Sleep* 44.1 (2021), zsaa161. DOI: 10.1093/sleep/zsaa161.
- [3] Raphael Vallat and Matthew P Walker. “An open-source, high-performance tool for automated sleep staging”. In: *eLife* 10 (Oct. 2021). Ed. by Adrien Peyrache, Christian BÄ́zchel, and Sophie Bagur, e70092. ISSN: 2050-084X. DOI: 10.7554/eLife.70092. URL: <https://doi.org/10.7554/eLife.70092>.